Project Coding – Backend source (Go)

Seminar Hall Booking System – representative .go sources (full repo on GitHub)

Full source code – This PDF lists representative files only (routing, auth, halls, requests, coordinator workflow). Admin/management handlers and other pages follow the same patterns. Repository: https://github.com/sreejithr247/seminar-hall-booking-system. **Commit:** 873a71a.

backend/main.go

```
package main

import (
	"log"
	"os"

	"seminar-hall-booking-system/internal/config"
	"seminar-hall-booking-system/internal/database"
	"seminar-hall-booking-system/internal/router"

	"github.com/gin-gonic/gin"
	"github.com/joho/godotenv"
)

func main() {
	// Load environment variables
	if err := godotenv.Load(); err != nil {
		log.Println("No .env file found, using environment variables")
	}

	// Load configuration
	cfg := config.Load()

	// Initialize database connection
	db, err := database.NewConnection(cfg.DatabaseURL)
	if err != nil {
		log.Fatalf("Failed to connect to database: %v", err)
	}
	defer db.Close()

	// Set Gin mode
	if cfg.Environment == "production" {
		gin.SetMode(gin.ReleaseMode)
	}

	// Initialize router
	r := router.SetupRouter(db, cfg)

	// Start server
	port := os.Getenv("PORT")
	if port == "" {
		port = "8080"
	}

	log.Printf("Server starting on port %s", port)
	if err := r.Run(":" + port); err != nil {
		log.Fatalf("Failed to start server: %v", err)
	}
}
```

backend/internal/router/router.go

```
package router

import (
	"seminar-hall-booking-system/internal/config"
	"seminar-hall-booking-system/internal/handlers"
	"seminar-hall-booking-system/internal/middleware"
	"seminar-hall-booking-system/internal/repositories"
	"seminar-hall-booking-system/internal/services"

	"github.com/gin-gonic/gin"
	"github.com/jackc/pgx/v5/pgxpool"
)

func SetupRouter(db *pgxpool.Pool, cfg *config.Config) *gin.Engine {
	r := gin.Default()

	// CORS middleware
```

```
	r.Use(middleware.CORS(cfg.FrontendURL))

	// Health check
	r.GET("/health", func(c *gin.Context) {
		c.JSON(200, gin.H{"status": "ok"})
	})

	// Initialize Repositories
	userRepo := repositories.NewUserRepository(db)
	hallRepo := repositories.NewHallRepository(db)
	requestRepo := repositories.NewRequestRepository(db)
	bookingRepo := repositories.NewBookingRepository(db)
	mgmtRepo := repositories.NewManagementRepository(db)

	// Initialize Services
	authService := services.NewAuthService(userRepo, cfg.JWTSecret)
	hallService := services.NewHallService(hallRepo)
	requestService := services.NewRequestService(requestRepo, hallRepo, userRepo, bookingRepo)
	coordinatorService := services.NewCoordinatorService(requestRepo, userRepo, bookingRepo)
	adminService := services.NewAdminService(requestRepo, bookingRepo)

	// Initialize Handlers
	authHandler := handlers.NewAuthHandler(authService, userRepo)
	hallHandler := handlers.NewHallHandler(hallService)
	requestHandler := handlers.NewRequestHandler(requestService)
	coordinatorHandler := handlers.NewCoordinatorHandler(coordinatorService)
	adminHandler := handlers.NewAdminHandler(adminService)
	mgmtHandler := handlers.NewManagementHandler(mgmtRepo)

	// API routes
	api := r.Group("/api")
	{
		// Public routes
		public := api.Group("")
		{
			public.GET("/halls", hallHandler.GetAllHalls)
			public.GET("/halls/:id", hallHandler.GetHallByID)
			public.GET("/availability", hallHandler.GetAvailability)
			public.GET("/departments", mgmtHandler.GetDepartments)
			public.GET("/clubs", mgmtHandler.GetClubs)
		}

		// Auth routes
		auth := api.Group("/auth")
		{
			auth.POST("/login", authHandler.Login)
			auth.POST("/logout", authHandler.Logout)

			protectedAuth := auth.Group("")
			protectedAuth.Use(middleware.AuthMiddleware(cfg.JWTSecret))
			protectedAuth.GET("/me", authHandler.GetMe)
		}

		// Protected routes
		protected := api.Group("")
		protected.Use(middleware.AuthMiddleware(cfg.JWTSecret))
		{
			// Requester routes
			requester := protected.Group("/requests")
			requester.Use(middleware.RequireRole("requester"))
			{
				requester.POST("", requestHandler.CreateRequest)
				requester.GET("/my", requestHandler.GetMyRequests)
			}

			// Dept Coordinator routes
			coordinator := protected.Group("")
			coordinator.Use(middleware.RequireRole("dept_coordinator"))
			{
				coordinator.GET("/requests/dept-pending",
coordinatorHandler.GetPendingRequests)
				coordinator.PATCH("/requests/:id/dept-review",
coordinatorHandler.ReviewRequest)
				coordinator.GET("/users/faculties", mgmtHandler.GetFaculties)
			}

			// Class management — accessible by both admin and dept_coordinator
			classGroup := protected.Group("")
```

```
                        classGroup.Use(middleware.RequireRole("admin", "dept_coordinator"))
                        {
                                classGroup.GET("/classes", mgmtHandler.GetClasses)
                                classGroup.POST("/classes", mgmtHandler.CreateClass)
                                classGroup.DELETE("/classes/:id", mgmtHandler.DeleteClass)
                        }

                        // Admin routes
                        admin := protected.Group("")
                        admin.Use(middleware.RequireRole("admin"))
                        {
                                // Request & Booking management
                                admin.GET("/requests/admin-pending", adminHandler.GetPendingRequests)
                                admin.PATCH("/requests/:id/admin-review", adminHandler.ReviewRequest)
                                admin.GET("/bookings", adminHandler.GetAllBookings)
                                admin.PATCH("/bookings/:id/cancel", mgmtHandler.CancelBooking)
                                // Department management
                                admin.POST("/departments", mgmtHandler.CreateDepartment)
                                admin.PUT("/departments/:id", mgmtHandler.UpdateDepartment)
                                admin.DELETE("/departments/:id", mgmtHandler.DeleteDepartment)
                                // Hall management
                                admin.POST("/halls", mgmtHandler.CreateHall)
                                admin.PUT("/halls/:id", mgmtHandler.UpdateHall)
                                admin.DELETE("/halls/:id", mgmtHandler.DeleteHall)
                                // User management
                                admin.GET("/users", mgmtHandler.GetAllUsers)
                                admin.POST("/users", mgmtHandler.CreateUser)
                                admin.DELETE("/users/:id", mgmtHandler.DeactivateUser)
                                admin.PATCH("/users/:id/reactivate", mgmtHandler.ReactivateUser)
                                // Club management (write-only; GET /clubs is public)
                                admin.POST("/clubs", mgmtHandler.CreateClub)
                                admin.DELETE("/clubs/:id", mgmtHandler.DeleteClub)
                                // Reports
                                admin.GET("/reports/hall-usage", mgmtHandler.GetHallUsageReport)
                                admin.GET("/reports/department-usage", mgmtHandler.GetDeptUsageReport)
                        }
                }
        }

        return r
}
```

backend/internal/config/config.go

```
package config

import (
        "os"
)

type Config struct {
        DatabaseURL  string
        JWTSecret    string
        Environment  string
        FrontendURL  string
        Port         string
}

func Load() *Config {
        return &Config{
                DatabaseURL: getEnv("DATABASE_URL", "postgresql://localhost:5432/seminar_hall_db?
sslmode=disable"),
                JWTSecret:    getEnv("JWT_SECRET", "your-secret-key-change-in-production"),
                Environment:  getEnv("ENVIRONMENT", "development"),
                FrontendURL:  getEnv("FRONTEND_URL", "http://localhost:3000"),
                Port:         getEnv("PORT", "8080"),
        }
}

func getEnv(key, defaultValue string) string {
        if value := os.Getenv(key); value != "" {
                return value
        }
        return defaultValue
}
```

backend/internal/database/database.go

```
package database

import (
	"context"
	"fmt"

	"github.com/jackc/pgx/v5"
	"github.com/jackc/pgx/v5/pgxpool"
)

// NewConnection creates a new PostgreSQL connection pool
func NewConnection(databaseURL string) (*pgxpool.Pool, error) {
	config, err := pgxpool.ParseConfig(databaseURL)
	if err != nil {
		return nil, fmt.Errorf("failed to parse database URL: %w", err)
	}

	// Explicitly set search_path to shbs on every new connection.
	// pgx does NOT honour the search_path URL parameter, so we must do it here.
	config.AfterConnect = func(ctx context.Context, conn *pgx.Conn) error {
		_, err := conn.Exec(ctx, "SET search_path TO shbs")
		return err
	}

	pool, err := pgxpool.NewWithConfig(context.Background(), config)
	if err != nil {
		return nil, fmt.Errorf("failed to create connection pool: %w", err)
	}

	// Test connection
	if err := pool.Ping(context.Background()); err != nil {
		return nil, fmt.Errorf("failed to ping database: %w", err)
	}

	return pool, nil
}
```

backend/internal/middleware/auth.go

```
package middleware

import (
	"net/http"
	"strings"

	"github.com/gin-gonic/gin"
	"github.com/golang-jwt/jwt/v5"
)

// AuthMiddleware validates JWT token from cookie or Authorization header
func AuthMiddleware(jwtSecret string) gin.HandlerFunc {
	return func(c *gin.Context) {
		// Try to get token from cookie first
		tokenString, err := c.Cookie("auth_token")
		if err != nil {
			// Try Authorization header
			authHeader := c.GetHeader("Authorization")
			if authHeader == "" {
				c.JSON(http.StatusUnauthorized, gin.H{"error": "Unauthorized"})
				c.Abort()
				return
			}

			// Extract token from "Bearer <token>"
			parts := strings.Split(authHeader, " ")
			if len(parts) != 2 || parts[0] != "Bearer" {
				c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid authorization
header"})
				c.Abort()
				return
			}
			tokenString = parts[1]
		}

		// Parse and validate token
		token, err := jwt.Parse(tokenString, func(token *jwt.Token) (interface{}, error) {
			if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
```

```
                        return nil, jwt.ErrSignatureInvalid
                }
                return []byte(jwtSecret), nil
        })

        if err != nil || !token.Valid {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid token"})
                c.Abort()
                return
        }

        // Extract claims
        if claims, ok := token.Claims.(jwt.MapClaims); ok {
                c.Set("user_id", int(claims["user_id"].(float64)))
                c.Set("username", claims["username"].(string))
                c.Set("role", claims["role"].(string))
        }

        c.Next()
    }
}

// RequireRole checks if user has one of the required roles
func RequireRole(allowedRoles ...string) gin.HandlerFunc {
    return func(c *gin.Context) {
        userRole, exists := c.Get("role")
        if !exists {
                c.JSON(http.StatusForbidden, gin.H{"error": "Insufficient permissions"})
                c.Abort()
                return
        }
        for _, r := range allowedRoles {
                if userRole.(string) == r {
                        c.Next()
                        return
                }
        }
        c.JSON(http.StatusForbidden, gin.H{"error": "Insufficient permissions"})
        c.Abort()
    }
}
```

backend/internal/middleware/cors.go

```
package middleware

import (
    "github.com/gin-gonic/gin"
)

// CORS middleware
func CORS(frontendURL string) gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Writer.Header().Set("Access-Control-Allow-Origin", frontendURL)
        c.Writer.Header().Set("Access-Control-Allow-Credentials", "true")
        c.Writer.Header().Set("Access-Control-Allow-Headers", "Content-Type, Content-Length,
Accept-Encoding, X-CSRF-Token, Authorization, accept, origin, Cache-Control, X-Requested-With")
        c.Writer.Header().Set("Access-Control-Allow-Methods", "POST, OPTIONS, GET, PUT, DELETE,
PATCH")

        if c.Request.Method == "OPTIONS" {
                c.AbortWithStatus(204)
                return
        }

        c.Next()
    }
}
```

backend/internal/models/user.go

```
package models

import (
    "time"
)

type UserRole string
```

```
const (
        RoleAdmin           UserRole = "admin"
        RoleDeptCoordinator UserRole = "dept_coordinator"
        RoleRequester       UserRole = "requester"
        RoleFaculty         UserRole = "faculty"
)

type User struct {
        UserID      int       `json:"user_id"`
        Username    string    `json:"username"`
        PasswordHash string   `json:"-"` // Never return in JSON
        FullName    string    `json:"full_name"`
        Email       *string   `json:"email"`
        Phone       *string   `json:"phone"`
        Role        UserRole  `json:"role"`
        DeptID      *int      `json:"dept_id"`
        IsActive    bool      `json:"is_active"`
        CreatedAt   time.Time `json:"created_at"`
        UpdatedAt   time.Time `json:"updated_at"`
}

type RequesterType string

const (
        RequesterTypeClass RequesterType = "class"
        RequesterTypeClub  RequesterType = "club"
)

type Requester struct {
        RequesterID   int           `json:"requester_id"`
        UserID        int           `json:"user_id"`
        RequesterType RequesterType `json:"requester_type"`
        ClassID       *int          `json:"class_id"`
        ClubID        *int          `json:"club_id"`
        CreatedAt     time.Time     `json:"created_at"`
}
```

backend/internal/models/booking.go

```
package models

import (
        "time"
)

type ApprovalStatus string

const (
        StatusPending   ApprovalStatus = "pending"
        StatusForwarded ApprovalStatus = "forwarded"
        StatusApproved  ApprovalStatus = "approved"
        StatusRejected  ApprovalStatus = "rejected"
        StatusAmended   ApprovalStatus = "amended"
)

type BookingStatus string

const (
        BookingStatusConfirmed BookingStatus = "confirmed"
        BookingStatusCancelled BookingStatus = "cancelled"
        BookingStatusCompleted BookingStatus = "completed"
        BookingStatusNoShow    BookingStatus = "no_show"
)

type Hall struct {
        HallID     int                    `json:"hall_id"`
        HallName   string                 `json:"hall_name"`
        Capacity   int                    `json:"capacity"`
        Location   *string                `json:"location"`
        Facilities map[string]interface{} `json:"facilities"`
        IsActive   bool                   `json:"is_active"`
}

type Request struct {
        RequestID         int            `json:"request_id"`
        RequesterID       int            `json:"requester_id"`
        HallID            int            `json:"hall_id"`
```

```
        EventTitle        string         `json:"event_title"`
        EventDescription  *string        `json:"event_description"`
        EventDate         time.Time      `json:"event_date"`
        StartTime         string         `json:"start_time"` // TIME format
        EndTime           string         `json:"end_time"`   // TIME format
        ExpectedAttendees *int           `json:"expected_attendees"`
        Purpose           *string        `json:"purpose"`
        DeptStatus        ApprovalStatus `json:"dept_status"`
        DeptApprovedBy    *int           `json:"dept_approved_by"`
        DeptApprovedAt    *time.Time     `json:"dept_approved_at"`
        DeptRemarks       *string        `json:"dept_remarks"`
        AdminStatus       ApprovalStatus `json:"admin_status"`
        AdminApprovedBy   *int           `json:"admin_approved_by"`
        AdminApprovedAt   *time.Time     `json:"admin_approved_at"`
        AdminRemarks      *string        `json:"admin_remarks"`
        RequestedAt       time.Time      `json:"requested_at"`
        IsCancelled       bool           `json:"is_cancelled"`
}

type Booking struct {
        BookingID   int           `json:"booking_id"`
        RequestID   int           `json:"request_id"`
        HallID      int           `json:"hall_id"`
        RequesterID int           `json:"requester_id"`
        EventTitle  string        `json:"event_title"`
        EventDate   time.Time     `json:"event_date"`
        StartTime   string        `json:"start_time"`
        EndTime     string        `json:"end_time"`
        Status      BookingStatus `json:"status"`
        CreatedAt   time.Time     `json:"created_at"`
        CompletedAt *time.Time    `json:"completed_at"`
}

type AvailabilitySlot struct {
        HallID   int       `json:"hall_id"`
        HallName string    `json:"hall_name"`
        Bookings []Booking `json:"bookings"`
}
```

backend/internal/models/management.go

```
package models

import "time"

type Department struct {
        DeptID    int       `json:"dept_id"`
        DeptName  string    `json:"dept_name"`
        DeptCode  *string   `json:"dept_code"`
        CreatedAt time.Time `json:"created_at"`
}

type Class struct {
        ClassID   int    `json:"class_id"`
        ClassName string `json:"class_name"`
        DeptID    int    `json:"dept_id"`
        Year      string `json:"year"`
}

type Club struct {
        ClubID      int     `json:"club_id"`
        ClubName    string  `json:"club_name"`
        DeptID      *int    `json:"dept_id"`
        Description *string `json:"description"`
}

// HallUsageReport – for reporting
type HallUsageReport struct {
        HallID    int    `json:"hall_id"`
        HallName  string `json:"hall_name"`
        TotalBookings int `json:"total_bookings"`
}

// DepartmentUsageReport – for reporting
type DepartmentUsageReport struct {
        DeptID        int    `json:"dept_id"`
        DeptName      string `json:"dept_name"`
```

```
        TotalRequests int    `json:"total_requests"`
}
```

backend/internal/handlers/auth_handler.go

```
package handlers

import (
        "net/http"
        "seminar-hall-booking-system/internal/repositories"
        "seminar-hall-booking-system/internal/services"

        "github.com/gin-gonic/gin"
)

type AuthHandler struct {
        authService *services.AuthService
        userRepo    *repositories.UserRepository
}

func NewAuthHandler(authService *services.AuthService, userRepo *repositories.UserRepository) *AuthHandler
{
        return &AuthHandler{
                authService: authService,
                userRepo:    userRepo,
        }
}

type LoginRequest struct {
        Username string `json:"username" binding:"required"`
        Password string `json:"password" binding:"required"`
}

// Login handles user authentication
func (h *AuthHandler) Login(c *gin.Context) {
        var req LoginRequest
        if err := c.ShouldBindJSON(&req); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        token, user, err := h.authService.Login(c.Request.Context(), req.Username, req.Password)
        if err != nil {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid credentials"})
                return
        }

        // Set HTTP-only cookie for the JWT
        c.SetCookie("auth_token", token, 3600*24*7, "/", "", false, true)

        c.JSON(http.StatusOK, gin.H{
                "message": "success",
                "token":   token,
                "user":    user,
        })
}

// Logout clears the auth cookie
func (h *AuthHandler) Logout(c *gin.Context) {
        c.SetCookie("auth_token", "", -1, "/", "", false, true)
        c.JSON(http.StatusOK, gin.H{"message": "logged out"})
}

// GetMe returns the authenticated user's details
func (h *AuthHandler) GetMe(c *gin.Context) {
        userID, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }

        // Fetch fresh user details from DB
        user, err := h.userRepo.GetByID(c.Request.Context(), userID.(int))
        if err != nil || user == nil {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "user not found"})
                return
        }

        // Check if user is a requester and fetch requester info
```

```
        // if user.Role == models.RoleRequester
        if user.Role == "requester" {
                reqInfo, err := h.userRepo.GetRequesterByUserID(c.Request.Context(), user.UserID)
                if err == nil && reqInfo != nil {
                        c.JSON(http.StatusOK, gin.H{"user": user, "requester": reqInfo})
                        return
                }
        }

        c.JSON(http.StatusOK, gin.H{"user": user})
}
```

backend/internal/handlers/request_handler.go

```
package handlers

import (
        "fmt"
        "net/http"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/services"
        "time"

        "github.com/gin-gonic/gin"
)

type RequestHandler struct {
        requestService *services.RequestService
}

func NewRequestHandler(requestService *services.RequestService) *RequestHandler {
        return &RequestHandler{requestService: requestService}
}

type CreateRequestPayload struct {
        HallID            int     `json:"hall_id" binding:"required"`
        EventTitle        string  `json:"event_title" binding:"required"`
        EventDescription  *string `json:"event_description"`
        EventDate         string  `json:"event_date" binding:"required"` // YYYY-MM-DD
        StartTime         string  `json:"start_time" binding:"required"` // HH:MM
        EndTime           string  `json:"end_time" binding:"required"`   // HH:MM
        ExpectedAttendees *int    `json:"expected_attendees"`
        Purpose           *string `json:"purpose"`
}

// CreateRequest creates a new booking request for a student/club
func (h *RequestHandler) CreateRequest(c *gin.Context) {
        userIDStr, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }
        userID := userIDStr.(int)

        var payload CreateRequestPayload
        if err := c.ShouldBindJSON(&payload); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        // Parse date
        eventDate, err := time.Parse("2006-01-02", payload.EventDate)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": "invalid event_date format (use YYYY-MM-
DD)"})
                return
        }

        // Format times string slightly (adding seconds if omitted by client)
        startTime := payload.StartTime
        if len(startTime) == 5 {
                startTime += ":00"
        }
        endTime := payload.EndTime
        if len(endTime) == 5 {
                endTime += ":00"
        }

        req := models.Request{
```

```
                HallID:            payload.HallID,
                EventTitle:        payload.EventTitle,
                EventDescription:  payload.EventDescription,
                EventDate:         eventDate,
                StartTime:         startTime,
                EndTime:           endTime,
                ExpectedAttendees: payload.ExpectedAttendees,
                Purpose:           payload.Purpose,
        }

        err = h.requestService.CreateRequest(c.Request.Context(), userID, &req)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        c.JSON(http.StatusCreated, gin.H{
                "message": "request submitted successfully",
                "data":    req,
        })
}

// GetMyRequests gets all requests made by the current authenticated requester
func (h *RequestHandler) GetMyRequests(c *gin.Context) {
        userIDStr, exists := c.Get("user_id")
        if !exists {
                fmt.Println("No user_id in context")
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }
        userID := userIDStr.(int)

        requests, err := h.requestService.GetMyRequests(c.Request.Context(), userID)
        if err != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
                return
        }

        // Make sure we never return null for a slice in JSON
        if requests == nil {
                requests = []models.Request{}
        }

        c.JSON(http.StatusOK, requests)
}
```

backend/internal/handlers/hall_handler.go

```
package handlers

import (
        "net/http"
        "seminar-hall-booking-system/internal/services"
        "strconv"

        "github.com/gin-gonic/gin"
)

type HallHandler struct {
        hallService *services.HallService
}

func NewHallHandler(hallService *services.HallService) *HallHandler {
        return &HallHandler{hallService: hallService}
}

// GetAllHalls returns a list of all active halls
func (h *HallHandler) GetAllHalls(c *gin.Context) {
        halls, err := h.hallService.GetAllHalls(c.Request.Context())
        if err != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to fetch halls"})
                return
        }

        c.JSON(http.StatusOK, halls)
}

// GetHallByID returns details for a single hall
func (h *HallHandler) GetHallByID(c *gin.Context) {
```

```
	idStr := c.Param("id")
	id, err := strconv.Atoi(idStr)
	if err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": "invalid hall id"})
		return
	}

	hall, err := h.hallService.GetHallByID(c.Request.Context(), id)
	if err != nil {
		c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to fetch hall details"})
		return
	}

	if hall == nil {
		c.JSON(http.StatusNotFound, gin.H{"error": "hall not found"})
		return
	}

	c.JSON(http.StatusOK, hall)
}

// GetAvailability returns bookings for a hall (if hall_id provided) or all halls (if hall_id omitted) on
a specific date
func (h *HallHandler) GetAvailability(c *gin.Context) {
	hallIDStr := c.Query("hall_id")
	dateStr := c.Query("date") // Format: YYYY-MM-DD
	startDateStr := c.Query("start_date")
	endDateStr := c.Query("end_date")

	if dateStr == "" && (startDateStr == "" || endDateStr == "") {
		c.JSON(http.StatusBadRequest, gin.H{"error": "date or (start_date and end_date) query
parameters are required"})
		return
	}

	// If hall_id is provided, return bookings for that hall ONLY (original behavior)
	if hallIDStr != "" {
		hallID, err := strconv.Atoi(hallIDStr)
		if err != nil {
			c.JSON(http.StatusBadRequest, gin.H{"error": "invalid hall_id"})
			return
		}

		// If range is provided
		if startDateStr != "" && endDateStr != "" {
			bookings, err := h.hallService.GetAvailabilityRange(c.Request.Context(), hallID,
startDateStr, endDateStr)
			if err != nil {
				c.JSON(http.StatusBadRequest, gin.H{"error": "Failed to fetch range
availability. Use YYYY-MM-DD."})
				return
			}
			c.JSON(http.StatusOK, bookings)
			return
		}

		// Fallback to single date
		bookings, err := h.hallService.GetAvailability(c.Request.Context(), hallID, dateStr)
		if err != nil {
			c.JSON(http.StatusBadRequest, gin.H{"error": "Failed to fetch availability. Use
YYYY-MM-DD."})
			return
		}
		c.JSON(http.StatusOK, bookings)
		return
	}

	// If hall_id is MISSING, return availability for ALL halls (new behavior)
	slots, err := h.hallService.GetAvailabilityForAll(c.Request.Context(), dateStr)
	if err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": "Failed to fetch availability. Use YYYY-MM-
DD."})
		return
	}

	c.JSON(http.StatusOK, slots)
}
```

backend/internal/handlers/coordinator_handler.go

```
package handlers

import (
	"net/http"
	"seminar-hall-booking-system/internal/models"
	"seminar-hall-booking-system/internal/services"
	"strconv"

	"github.com/gin-gonic/gin"
)

type CoordinatorHandler struct {
	coordinatorService *services.CoordinatorService
}

func NewCoordinatorHandler(coordinatorService *services.CoordinatorService) *CoordinatorHandler {
	return &CoordinatorHandler{coordinatorService: coordinatorService}
}

// GetPendingRequests gets requests waiting for department approval
func (h *CoordinatorHandler) GetPendingRequests(c *gin.Context) {
	userIDStr, exists := c.Get("user_id")
	if !exists {
		c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
		return
	}
	userID := userIDStr.(int)

	requests, err := h.coordinatorService.GetDepartmentPendingRequests(c.Request.Context(), userID)
	if err != nil {
		c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
		return
	}

	if requests == nil {
		requests = []models.Request{}
	}

	c.JSON(http.StatusOK, requests)
}

type ReviewRequestPayload struct {
	Action  string  `json:"action" binding:"required"` // "approve" or "reject"
	Remarks *string `json:"remarks"`
}

// ReviewRequest approves or rejects a department request
func (h *CoordinatorHandler) ReviewRequest(c *gin.Context) {
	userIDStr, exists := c.Get("user_id")
	if !exists {
		c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
		return
	}
	userID := userIDStr.(int)

	requestIDStr := c.Param("id")
	requestID, err := strconv.Atoi(requestIDStr)
	if err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": "invalid request id"})
		return
	}

	var payload ReviewRequestPayload
	if err := c.ShouldBindJSON(&payload); err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
		return
	}

	err = h.coordinatorService.ReviewRequest(c.Request.Context(), userID, requestID, payload.Action,
payload.Remarks)
	if err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
		return
	}

	c.JSON(http.StatusOK, gin.H{"message": "request " + payload.Action + " successfully"})
}
```

backend/internal/services/auth_service.go

```
package services

import (
	"context"
	"errors"
	"seminar-hall-booking-system/internal/models"
	"seminar-hall-booking-system/internal/repositories"
	"time"

	"github.com/golang-jwt/jwt/v5"
	"golang.org/x/crypto/bcrypt"
)

type AuthService struct {
	userRepo  *repositories.UserRepository
	jwtSecret string
}

func NewAuthService(userRepo *repositories.UserRepository, jwtSecret string) *AuthService {
	return &AuthService{
		userRepo:  userRepo,
		jwtSecret: jwtSecret,
	}
}

func (s *AuthService) Login(ctx context.Context, username, password string) (string, *models.User, error)
{
	user, err := s.userRepo.GetByUsername(ctx, username)
	if err != nil {
		return "", nil, err
	}
	if user == nil {
		return "", nil, errors.New("invalid credentials")
	}

	// Attempt bcrypt comparison
	err = bcrypt.CompareHashAndPassword([]byte(user.PasswordHash), []byte(password))
	if err != nil {
		// Fallback for sample DB data which might be plain text or pseudo-hashed strings like
"hashedpass_admin"
		if user.PasswordHash != password {
			return "", nil, errors.New("invalid credentials")
		}
	}

	// Generate JWT claims
	token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
		"user_id":  user.UserID,
		"username": user.Username,
		"role":     user.Role,
		"exp":      time.Now().Add(time.Hour * 24 * 7).Unix(), // 7 days expiration
	})

	// Sign token
	tokenString, err := token.SignedString([]byte(s.jwtSecret))
	if err != nil {
		return "", nil, err
	}

	return tokenString, user, nil
}
```

backend/internal/services/request_service.go

```
package services

import (
	"context"
	"errors"
	"seminar-hall-booking-system/internal/models"
	"seminar-hall-booking-system/internal/repositories"
	"time"
)

type RequestService struct {
	requestRepo *repositories.RequestRepository
	hallRepo    *repositories.HallRepository
	userRepo    *repositories.UserRepository
```

```
        bookingRepo *repositories.BookingRepository
}

func NewRequestService(
        reqRepo *repositories.RequestRepository,
        hallRepo *repositories.HallRepository,
        userRepo *repositories.UserRepository,
        bookingRepo *repositories.BookingRepository,
) *RequestService {
        return &RequestService{
                requestRepo: reqRepo,
                hallRepo:    hallRepo,
                userRepo:    userRepo,
                bookingRepo: bookingRepo,
        }
}

// CreateRequest handles the business logic for submitting a new seminar hall request
func (s *RequestService) CreateRequest(ctx context.Context, userID int, req *models.Request) error {
        // 1. Verify user is a requester
        requester, err := s.userRepo.GetRequesterByUserID(ctx, userID)
        if err != nil {
                return err
        }
        if requester == nil {
                return errors.New("user is not registered as a requester (class or club)")
        }

        // 2. Validate basic rules
        if req.ExpectedAttendees != nil && *req.ExpectedAttendees <= 0 {
                return errors.New("expected attendees must be greater than zero")
        }

        // 3. Verify Hall exists and capacity is sufficient
        hall, err := s.hallRepo.GetByID(ctx, req.HallID)
        if err != nil {
                return err
        }
        if hall == nil {
                return errors.New("hall not found")
        }
        if req.ExpectedAttendees != nil && *req.ExpectedAttendees > hall.Capacity {
                return errors.New("expected attendees exceeds hall capacity")
        }

        // 4. Validate time (end_time > start_time)
        startTime, err := time.Parse("15:04:05", req.StartTime)
        if err != nil {
                return errors.New("invalid start_time format, expected HH:MM:SS")
        }
        endTime, err := time.Parse("15:04:05", req.EndTime)
        if err != nil {
                return errors.New("invalid end_time format, expected HH:MM:SS")
        }
        if !endTime.After(startTime) {
                return errors.New("end time must be strictly after start time")
        }

        // 5. Check for overlapping bookings before placing the request
        overlap, err := s.bookingRepo.CheckOverlap(ctx, req.HallID, req.EventDate, req.StartTime,
req.EndTime)
        if err != nil {
                return err
        }
        if overlap {
                return errors.New("cannot place request: this time slot overlaps with an existing
confirmed booking")
        }

        // 6. Build and save request
        req.RequesterID = requester.RequesterID

        return s.requestRepo.Create(ctx, req)
}

func (s *RequestService) GetMyRequests(ctx context.Context, userID int) ([]models.Request, error) {
        requester, err := s.userRepo.GetRequesterByUserID(ctx, userID)
        if err != nil {
```

```
                return nil, err
        }
        if requester == nil {
                return nil, errors.New("user is not a requester")
        }

        return s.requestRepo.GetByRequester(ctx, requester.RequesterID)
}
```

backend/internal/services/hall_service.go

```
package services

import (
        "context"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/repositories"
        "time"
)

type HallService struct {
        hallRepo *repositories.HallRepository
}

func NewHallService(hallRepo *repositories.HallRepository) *HallService {
        return &HallService{hallRepo: hallRepo}
}

func (s *HallService) GetAllHalls(ctx context.Context) ([]models.Hall, error) {
        return s.hallRepo.GetAll(ctx)
}

func (s *HallService) GetHallByID(ctx context.Context, id int) (*models.Hall, error) {
        return s.hallRepo.GetByID(ctx, id)
}

func (s *HallService) GetAvailability(ctx context.Context, hallID int, dateStr string) ([]models.Booking,
error) {
        // Parse date string (YYYY-MM-DD)
        date, err := time.Parse("2006-01-02", dateStr)
        if err != nil {
                return nil, err
        }

        return s.hallRepo.GetAvailability(ctx, hallID, date)
}

func (s *HallService) GetAvailabilityForAll(ctx context.Context, dateStr string)
([]models.AvailabilitySlot, error) {
        // Parse date string (YYYY-MM-DD)
        date, err := time.Parse("2006-01-02", dateStr)
        if err != nil {
                return nil, err
        }

        return s.hallRepo.GetAvailabilityForAll(ctx, date)
}

func (s *HallService) GetAvailabilityRange(ctx context.Context, hallID int, startDateStr string,
endDateStr string) ([]models.Booking, error) {
        startDate, err := time.Parse("2006-01-02", startDateStr)
        if err != nil {
                return nil, err
        }
        endDate, err := time.Parse("2006-01-02", endDateStr)
        if err != nil {
                return nil, err
        }

        return s.hallRepo.GetAvailabilityRange(ctx, hallID, startDate, endDate)
}
```

backend/internal/services/coordinator_service.go

```
package services

import (
        "context"
        "errors"
        "seminar-hall-booking-system/internal/models"
```

```
	"seminar-hall-booking-system/internal/repositories"
)

type CoordinatorService struct {
	requestRepo *repositories.RequestRepository
	userRepo    *repositories.UserRepository
	bookingRepo *repositories.BookingRepository
}

func NewCoordinatorService(reqRepo *repositories.RequestRepository, userRepo *repositories.UserRepository,
bookingRepo *repositories.BookingRepository) *CoordinatorService {
	return &CoordinatorService{
		requestRepo: reqRepo,
		userRepo:    userRepo,
		bookingRepo: bookingRepo,
	}
}

// GetDepartmentPendingRequests gets all pending requests for the coordinator's department
func (s *CoordinatorService) GetDepartmentPendingRequests(ctx context.Context, coordinatorID int)
([]models.Request, error) {
	coordinator, err := s.userRepo.GetByID(ctx, coordinatorID)
	if err != nil {
		return nil, err
	}
	if coordinator == nil || coordinator.DeptID == nil {
		return nil, errors.New("coordinator department not found")
	}

	// For a complete implementation, RequestRepository needs a GetByDepartmentAndStatus method.
	// We'll implement a workaround here by fetching all requests and filtering in memory or ideally
updating the repo.
	// Assuming an update to RequestRepository:
	return s.requestRepo.GetByDepartmentAndStatus(ctx, *coordinator.DeptID, models.StatusPending)
}

// ReviewRequest allows the coordinator to approve (forward) or reject a request
func (s *CoordinatorService) ReviewRequest(ctx context.Context, coordinatorID int, requestID int, action
string, remarks *string) error {
	// 1. Verify coordinator
	coordinator, err := s.userRepo.GetByID(ctx, coordinatorID)
	if err != nil || coordinator == nil || coordinator.DeptID == nil {
		return errors.New("invalid coordinator")
	}

	// 2. Verify request exists and is pending
	req, err := s.requestRepo.GetByID(ctx, requestID)
	if err != nil || req == nil {
		return errors.New("request not found")
	}

	if req.DeptStatus != models.StatusPending {
		return errors.New("request is no longer pending department approval")
	}

	// In a full implementation, verify the request belongs to the coordinator's department here by
looking up the requester's department.

	// 3. Update status
	var newStatus models.ApprovalStatus
	if action == "approve" {
		// Before forwarding, check if there's an existing overlapping booking
		overlap, err := s.bookingRepo.CheckOverlap(ctx, req.HallID, req.EventDate, req.StartTime,
req.EndTime)
		if err != nil {
			return err
		}
		if overlap {
			return errors.New("cannot forward: this request overlaps with an existing
confirmed booking")
		}
		newStatus = models.StatusForwarded
	} else if action == "reject" {
		newStatus = models.StatusRejected
	} else {
		return errors.New("invalid action. Must be 'approve' or 'reject'")
	}
```

```
        return s.requestRepo.UpdateDeptStatus(ctx, requestID, newStatus, coordinatorID, remarks)
}
```

backend/internal/repositories/user_repository.go

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"

        "github.com/jackc/pgx/v5"
        "github.com/jackc/pgx/v5/pgxpool"
)

type UserRepository struct {
        db *pgxpool.Pool
}

func NewUserRepository(db *pgxpool.Pool) *UserRepository {
        return &UserRepository{db: db}
}

func (r *UserRepository) GetByUsername(ctx context.Context, username string) (*models.User, error) {
        query := `
                SELECT user_id, username, password_hash, full_name, email, phone, role, dept_id,
is_active, created_at, updated_at
                FROM users
                WHERE username = $1 AND is_active = true
        `

        var user models.User
        err := r.db.QueryRow(ctx, query, username).Scan(
                &user.UserID,
                &user.Username,
                &user.PasswordHash,
                &user.FullName,
                &user.Email,
                &user.Phone,
                &user.Role,
                &user.DeptID,
                &user.IsActive,
                &user.CreatedAt,
                &user.UpdatedAt,
        )

        if err != nil {
                if err == pgx.ErrNoRows {
                        return nil, nil // Not found
                }
                return nil, err
        }

        return &user, nil
}

func (r *UserRepository) GetByID(ctx context.Context, id int) (*models.User, error) {
        query := `
                SELECT user_id, username, password_hash, full_name, email, phone, role, dept_id,
is_active, created_at, updated_at
                FROM users
                WHERE user_id = $1 AND is_active = true
        `

        var user models.User
        err := r.db.QueryRow(ctx, query, id).Scan(
                &user.UserID,
                &user.Username,
                &user.PasswordHash,
                &user.FullName,
                &user.Email,
                &user.Phone,
                &user.Role,
                &user.DeptID,
                &user.IsActive,
                &user.CreatedAt,
                &user.UpdatedAt,
        )
```

```
        if err != nil {
                if err == pgx.ErrNoRows {
                        return nil, nil
                }
                return nil, err
        }

        return &user, nil
}

func (r *UserRepository) GetRequesterByUserID(ctx context.Context, userID int) (*models.Requester, error)
{
        query := `
                SELECT requester_id, user_id, requester_type, class_id, club_id, created_at
                FROM requesters
                WHERE user_id = $1
        `

        var req models.Requester
        err := r.db.QueryRow(ctx, query, userID).Scan(
                &req.RequesterID,
                &req.UserID,
                &req.RequesterType,
                &req.ClassID,
                &req.ClubID,
                &req.CreatedAt,
        )

        if err != nil {
                if err == pgx.ErrNoRows {
                        return nil, nil
                }
                return nil, err
        }

        return &req, nil
}
```

backend/internal/repositories/request_repository.go

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"

        "github.com/jackc/pgx/v5"
        "github.com/jackc/pgx/v5/pgxpool"
)

type RequestRepository struct {
        db *pgxpool.Pool
}

func NewRequestRepository(db *pgxpool.Pool) *RequestRepository {
        return &RequestRepository{db: db}
}

// Create new request
func (r *RequestRepository) Create(ctx context.Context, req *models.Request) error {
        query := `
                INSERT INTO requests (requester_id, hall_id, event_title, event_description, event_date,
start_time, end_time, expected_attendees, purpose)
                VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
                RETURNING request_id, dept_status, admin_status, requested_at
        `

        err := r.db.QueryRow(ctx, query,
                req.RequesterID,
                req.HallID,
                req.EventTitle,
                req.EventDescription,
                req.EventDate,
                req.StartTime,
                req.EndTime,
                req.ExpectedAttendees,
                req.Purpose,
        ).Scan(&req.RequestID, &req.DeptStatus, &req.AdminStatus, &req.RequestedAt)
```

```
        return err
}

// GetByID gets a specific request
func (r *RequestRepository) GetByID(ctx context.Context, id int) (*models.Request, error) {
        query := `
                SELECT request_id, requester_id, hall_id, event_title, event_description, event_date,
start_time, end_time,
                expected_attendees, purpose, dept_status, dept_approved_by, dept_approved_at,
dept_remarks,
                admin_status, admin_approved_by, admin_approved_at, admin_remarks, requested_at,
is_cancelled
                FROM requests
                WHERE request_id = $1
        `

        var req models.Request
        err := r.db.QueryRow(ctx, query, id).Scan(
                &req.RequestID,
                &req.RequesterID,
                &req.HallID,
                &req.EventTitle,
                &req.EventDescription,
                &req.EventDate,
                &req.StartTime,
                &req.EndTime,
                &req.ExpectedAttendees,
                &req.Purpose,
                &req.DeptStatus,
                &req.DeptApprovedBy,
                &req.DeptApprovedAt,
                &req.DeptRemarks,
                &req.AdminStatus,
                &req.AdminApprovedBy,
                &req.AdminApprovedAt,
                &req.AdminRemarks,
                &req.RequestedAt,
                &req.IsCancelled,
        )

        if err != nil {
                if err == pgx.ErrNoRows {
                        return nil, nil // Not found
                }
                return nil, err
        }

        return &req, nil
}

// GetByRequester gets all requests for a specific requester
func (r *RequestRepository) GetByRequester(ctx context.Context, requesterID int) ([]models.Request, error)
{
        query := `
                SELECT request_id, requester_id, hall_id, event_title, event_description, event_date,
start_time, end_time,
                expected_attendees, purpose, dept_status, dept_approved_by, dept_approved_at,
dept_remarks,
                admin_status, admin_approved_by, admin_approved_at, admin_remarks, requested_at,
is_cancelled
                FROM requests
                WHERE requester_id = $1
                ORDER BY requested_at DESC
        `

        rows, err := r.db.Query(ctx, query, requesterID)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var requests []models.Request
        for rows.Next() {
                var req models.Request
                err := rows.Scan(
                        &req.RequestID,
                        &req.RequesterID,
                        &req.HallID,
```

```
                        &req.EventTitle,
                        &req.EventDescription,
                        &req.EventDate,
                        &req.StartTime,
                        &req.EndTime,
                        &req.ExpectedAttendees,
                        &req.Purpose,
                        &req.DeptStatus,
                        &req.DeptApprovedBy,
                        &req.DeptApprovedAt,
                        &req.DeptRemarks,
                        &req.AdminStatus,
                        &req.AdminApprovedBy,
                        &req.AdminApprovedAt,
                        &req.AdminRemarks,
                        &req.RequestedAt,
                        &req.IsCancelled,
                )
                if err != nil {
                        return nil, err
                }
                requests = append(requests, req)
        }

        return requests, nil
}

// UpdateDeptStatus updates the dept status
func (r *RequestRepository) UpdateDeptStatus(ctx context.Context, reqID int, status models.ApprovalStatus,
byUserID int, remarks *string) error {
        query := `
                UPDATE requests
                SET dept_status = $1, dept_approved_by = $2, dept_approved_at = NOW(), dept_remarks = $3
                WHERE request_id = $4
        `
        _, err := r.db.Exec(ctx, query, status, byUserID, remarks, reqID)
        return err
}

// UpdateAdminStatus updates the admin status
func (r *RequestRepository) UpdateAdminStatus(ctx context.Context, reqID int, status
models.ApprovalStatus, byUserID int, remarks *string) error {
        query := `
                UPDATE requests
                SET admin_status = $1, admin_approved_by = $2, admin_approved_at = NOW(), admin_remarks =
$3
                WHERE request_id = $4
        `
        _, err := r.db.Exec(ctx, query, status, byUserID, remarks, reqID)
        return err
}

// GetByDepartmentAndStatus gets pending requests for a specific department
func (r *RequestRepository) GetByDepartmentAndStatus(ctx context.Context, deptID int, status
models.ApprovalStatus) ([]models.Request, error) {
        // A bit tricky because we have to join with requesters -> users to find the dept_id
        query := `
                SELECT r.request_id, r.requester_id, r.hall_id, r.event_title, r.event_description,
r.event_date, r.start_time, r.end_time,
                r.expected_attendees, r.purpose, r.dept_status, r.dept_approved_by, r.dept_approved_at,
r.dept_remarks,
                r.admin_status, r.admin_approved_by, r.admin_approved_at, r.admin_remarks, r.requested_at,
r.is_cancelled
                FROM requests r
                JOIN requesters req ON r.requester_id = req.requester_id
                JOIN users u ON req.user_id = u.user_id
                WHERE u.dept_id = $1 AND r.dept_status = $2
                ORDER BY r.requested_at ASC
        `

        rows, err := r.db.Query(ctx, query, deptID, status)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var requests []models.Request
        for rows.Next() {
```

```
                var req models.Request
                err := rows.Scan(
                        &req.RequestID,
                        &req.RequesterID,
                        &req.HallID,
                        &req.EventTitle,
                        &req.EventDescription,
                        &req.EventDate,
                        &req.StartTime,
                        &req.EndTime,
                        &req.ExpectedAttendees,
                        &req.Purpose,
                        &req.DeptStatus,
                        &req.DeptApprovedBy,
                        &req.DeptApprovedAt,
                        &req.DeptRemarks,
                        &req.AdminStatus,
                        &req.AdminApprovedBy,
                        &req.AdminApprovedAt,
                        &req.AdminRemarks,
                        &req.RequestedAt,
                        &req.IsCancelled,
                )
                if err != nil {
                        return nil, err
                }
                requests = append(requests, req)
        }
        return requests, nil
}

// GetByAdminStatus gets requests that have a certain admin status (and dept status)
func (r *RequestRepository) GetByAdminStatus(ctx context.Context, adminStatus models.ApprovalStatus,
deptStatus models.ApprovalStatus) ([]models.Request, error) {
        query := `
                SELECT request_id, requester_id, hall_id, event_title, event_description, event_date,
start_time, end_time,
                expected_attendees, purpose, dept_status, dept_approved_by, dept_approved_at,
dept_remarks,
                admin_status, admin_approved_by, admin_approved_at, admin_remarks, requested_at,
is_cancelled
                FROM requests
                WHERE admin_status = $1 AND dept_status = $2
                ORDER BY requested_at ASC
        `

        rows, err := r.db.Query(ctx, query, adminStatus, deptStatus)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var requests []models.Request
        for rows.Next() {
                var req models.Request
                err := rows.Scan(
                        &req.RequestID,
                        &req.RequesterID,
                        &req.HallID,
                        &req.EventTitle,
                        &req.EventDescription,
                        &req.EventDate,
                        &req.StartTime,
                        &req.EndTime,
                        &req.ExpectedAttendees,
                        &req.Purpose,
                        &req.DeptStatus,
                        &req.DeptApprovedBy,
                        &req.DeptApprovedAt,
                        &req.DeptRemarks,
                        &req.AdminStatus,
                        &req.AdminApprovedBy,
                        &req.AdminApprovedAt,
                        &req.AdminRemarks,
                        &req.RequestedAt,
                        &req.IsCancelled,
                )
                if err != nil {
```

```
                        return nil, err
                }
                requests = append(requests, req)
        }
        return requests, nil
}
```

backend/internal/repositories/booking_repository.go

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"
        "time"

        "github.com/jackc/pgx/v5/pgxpool"
)

type BookingRepository struct {
        db *pgxpool.Pool
}

func NewBookingRepository(db *pgxpool.Pool) *BookingRepository {
        return &BookingRepository{db: db}
}

// Create inserts a new booking (trigger actively checks for overlap)
func (r *BookingRepository) Create(ctx context.Context, b *models.Booking) error {
        query := `
                INSERT INTO bookings (request_id, hall_id, requester_id, event_title, event_date,
start_time, end_time)
                VALUES ($1, $2, $3, $4, $5, $6, $7)
                RETURNING booking_id, status, created_at
        `

        err := r.db.QueryRow(ctx, query,
                b.RequestID,
                b.HallID,
                b.RequesterID,
                b.EventTitle,
                b.EventDate,
                b.StartTime,
                b.EndTime,
        ).Scan(&b.BookingID, &b.Status, &b.CreatedAt)

        return err
}

// GetAll returns all bookings
func (r *BookingRepository) GetAll(ctx context.Context) ([]models.Booking, error) {
        query := `
                SELECT booking_id, request_id, hall_id, requester_id, event_title, event_date, start_time,
end_time, status, created_at, completed_at
                FROM bookings
                ORDER BY event_date ASC, start_time ASC
        `

        rows, err := r.db.Query(ctx, query)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var bookings []models.Booking
        for rows.Next() {
                var b models.Booking
                err := rows.Scan(
                        &b.BookingID,
                        &b.RequestID,
                        &b.HallID,
                        &b.RequesterID,
                        &b.EventTitle,
                        &b.EventDate,
                        &b.StartTime,
                        &b.EndTime,
                        &b.Status,
                        &b.CreatedAt,
```

```
                        &b.CompletedAt,
                )
                if err != nil {
                        return nil, err
                }
                bookings = append(bookings, b)
        }

        return bookings, nil
}

// GetByRequester returns all bookings for a specific requester
func (r *BookingRepository) GetByRequester(ctx context.Context, requesterID int) ([]models.Booking, error)
{
        query := `
                SELECT booking_id, request_id, hall_id, requester_id, event_title, event_date, start_time,
end_time, status, created_at, completed_at
                FROM bookings
                WHERE requester_id = $1
                ORDER BY event_date DESC
        `

        rows, err := r.db.Query(ctx, query, requesterID)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var bookings []models.Booking
        for rows.Next() {
                var b models.Booking
                err := rows.Scan(
                        &b.BookingID,
                        &b.RequestID,
                        &b.HallID,
                        &b.RequesterID,
                        &b.EventTitle,
                        &b.EventDate,
                        &b.StartTime,
                        &b.EndTime,
                        &b.Status,
                        &b.CreatedAt,
                        &b.CompletedAt,
                )
                if err != nil {
                        return nil, err
                }
                bookings = append(bookings, b)
        }

        return bookings, nil
}

// UpdateStatus allows changing a booking's status
func (r *BookingRepository) UpdateStatus(ctx context.Context, bookingID int, status models.BookingStatus)
error {
        query := `UPDATE bookings SET status = $1 WHERE booking_id = $2`
        _, err := r.db.Exec(ctx, query, status, bookingID)
        return err
}

// CheckOverlap returns true if there's an existing active booking for the same hall and time
func (r *BookingRepository) CheckOverlap(ctx context.Context, hallID int, date time.Time, startTime
string, endTime string) (bool, error) {
        query := `
                SELECT COUNT(*)
                FROM bookings
                WHERE hall_id = $1
                  AND event_date = $2
                  AND status != 'cancelled'
                  AND (
                          (start_time < $4 AND end_time > $3)
                  )
        `
        var count int
        // We use QueryRow to get the count of overlapping bookings
        err := r.db.QueryRow(ctx, query, hallID, date, startTime, endTime).Scan(&count)
        if err != nil {
```

```
                return false, err
        }
        return count > 0, nil
}
```

backend/internal/repositories/hall_repository.go

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"
        "time"

        "github.com/jackc/pgx/v5"
        "github.com/jackc/pgx/v5/pgxpool"
)

type HallRepository struct {
        db *pgxpool.Pool
}

func NewHallRepository(db *pgxpool.Pool) *HallRepository {
        return &HallRepository{db: db}
}

func (r *HallRepository) GetAll(ctx context.Context) ([]models.Hall, error) {
        query := `
                SELECT hall_id, hall_name, capacity, location, facilities, is_active
                FROM halls
                WHERE is_active = true
        `

        rows, err := r.db.Query(ctx, query)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        halls := []models.Hall{}
        for rows.Next() {
                var hall models.Hall
                err := rows.Scan(
                        &hall.HallID,
                        &hall.HallName,
                        &hall.Capacity,
                        &hall.Location,
                        &hall.Facilities,
                        &hall.IsActive,
                )
                if err != nil {
                        return nil, err
                }
                halls = append(halls, hall)
        }

        return halls, nil
}

func (r *HallRepository) GetByID(ctx context.Context, id int) (*models.Hall, error) {
        query := `
                SELECT hall_id, hall_name, capacity, location, facilities, is_active
                FROM halls
                WHERE hall_id = $1 AND is_active = true
        `

        var hall models.Hall
        err := r.db.QueryRow(ctx, query, id).Scan(
                &hall.HallID,
                &hall.HallName,
                &hall.Capacity,
                &hall.Location,
                &hall.Facilities,
                &hall.IsActive,
        )

        if err != nil {
                if err == pgx.ErrNoRows {
                        return nil, nil
```

```
                }
                return nil, err
        }

        return &hall, nil
}

// GetAvailability gets all confirmed bookings for a specific hall on a specific date
func (r *HallRepository) GetAvailability(ctx context.Context, hallID int, date time.Time)
([]models.Booking, error) {
        query := `
                SELECT booking_id, request_id, hall_id, requester_id, event_title, event_date, start_time,
end_time, status, created_at, completed_at
                FROM bookings
                WHERE hall_id = $1 AND event_date = $2 AND status != 'cancelled'
                ORDER BY start_time ASC
        `

        rows, err := r.db.Query(ctx, query, hallID, date.Format("2006-01-02"))
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        bookings := []models.Booking{}
        for rows.Next() {
                var b models.Booking
                err := rows.Scan(
                        &b.BookingID,
                        &b.RequestID,
                        &b.HallID,
                        &b.RequesterID,
                        &b.EventTitle,
                        &b.EventDate,
                        &b.StartTime,
                        &b.EndTime,
                        &b.Status,
                        &b.CreatedAt,
                        &b.CompletedAt,
                )
                if err != nil {
                        return nil, err
                }
                bookings = append(bookings, b)
        }

        return bookings, nil
}

// GetAvailabilityRange gets all confirmed bookings for a specific hall between two dates
func (r *HallRepository) GetAvailabilityRange(ctx context.Context, hallID int, startDate time.Time,
endDate time.Time) ([]models.Booking, error) {
        query := `
                SELECT booking_id, request_id, hall_id, requester_id, event_title, event_date, start_time,
end_time, status, created_at, completed_at
                FROM bookings
                WHERE hall_id = $1 AND event_date BETWEEN $2 AND $3 AND status != 'cancelled'
                ORDER BY event_date ASC, start_time ASC
        `

        rows, err := r.db.Query(ctx, query, hallID, startDate.Format("2006-01-02"), endDate.Format("2006-
01-02"))
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        bookings := []models.Booking{}
        for rows.Next() {
                var b models.Booking
                err := rows.Scan(
                        &b.BookingID, &b.RequestID, &b.HallID, &b.RequesterID,
                        &b.EventTitle, &b.EventDate, &b.StartTime, &b.EndTime,
                        &b.Status, &b.CreatedAt, &b.CompletedAt,
                )
                if err != nil {
                        return nil, err
                }
```

```
                bookings = append(bookings, b)
        }

        return bookings, nil
}
// GetAvailabilityForAll gets confirmed bookings for ALL active halls on a specific date
func (r *HallRepository) GetAvailabilityForAll(ctx context.Context, date time.Time)
([]models.AvailabilitySlot, error) {
        // 1. Get all active halls
        halls, err := r.GetAll(ctx)
        if err != nil {
                return nil, err
        }

        // 2. Get all bookings for that date
        query := `
                SELECT booking_id, request_id, hall_id, requester_id, event_title, event_date, start_time,
end_time, status, created_at, completed_at
                FROM bookings
                WHERE event_date = $1 AND status != 'cancelled'
                ORDER BY start_time ASC
        `
        rows, err := r.db.Query(ctx, query, date.Format("2006-01-02"))
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        // 3. Group bookings by hall_id
        bookingsByHall := make(map[int][]models.Booking)
        for rows.Next() {
                var b models.Booking
                err := rows.Scan(
                        &b.BookingID, &b.RequestID, &b.HallID, &b.RequesterID,
                        &b.EventTitle, &b.EventDate, &b.StartTime, &b.EndTime,
                        &b.Status, &b.CreatedAt, &b.CompletedAt,
                )
                if err != nil {
                        return nil, err
                }
                bookingsByHall[b.HallID] = append(bookingsByHall[b.HallID], b)
        }

        // 4. Map halls to availability slots
        var slots []models.AvailabilitySlot
        for _, h := range halls {
                bookings := bookingsByHall[h.HallID]
                if bookings == nil {
                        bookings = []models.Booking{}
                }
                slots = append(slots, models.AvailabilitySlot{
                        HallID:   h.HallID,
                        HallName: h.HallName,
                        Bookings: bookings,
                })
        }

        return slots, nil
}
```